

ALGORITMOS E PENSAMENTO COMPUTACIONAL



Funções e Modularização

- Definição e chamada de funções
- Parâmetros: passagem por valor e passagem por referência
- Retorno de valores
- Estruturas de dados heterogêneas (structs)

Professor Marco Antonio

2º semestre 2026

Calendário de avaliações



Atenção!!!

Todas as avaliações serão realizadas **sem o uso de computador**!!!!

Instrumento	Data	Valor	Observação
Exercícios de aula	Até 16/11	Até 1,0 (0,6 entregas de aula + 0,4 Github)	Individual
Projetos	Até 16/11	1,0 (0,25 cada)	Em equipes (3 alunos)
Avaliação Parcial	09/11	3,0	Individual
Avaliação Regimental	30/11	5,0	Individual
Avaliação Final	14/12	5,0	Individual

Reflexão inicial

Um determinado sistema precisa calcular o valor final de diversos produtos.
Para cada produto teremos:

vendas.c

```
1  // Calcula o valor final
2  ...
3  valor_final = preco -
       preco * desconto / 100.0;
4  ...
```

carrinho.c

```
1  // Aplica desconto no item
2  ...
3  valor_final = preco -
       preco * desconto / 100.0;
4  ...
```

MESMA FÓRMULA EM VÁRIOS LUGARES

relatorio.c

```
1  // Calcula total com desconto
2  ...
3  valor_final = preco -
       preco * desconto / 100.0;
4  ...
```

estoque.c

```
1  // Atualiza preço com desconto
2  ...
3  valor_final = preco -
       preco * desconto / 100.0;
4  ...
```



A mesma fórmula aparece em diferentes partes do programa.

Discuta

1. O que acontece se a regra de desconto mudar?
2. Quantos lugares precisarão ser alterados?
3. E se esquecermos de alterar uma das cópias?
4. Como poderíamos escrever essa regra apenas uma vez?

Conceitos abordados nesta aula

Imagine um sistema acadêmico completo:



Modularização

Modularizar significa organizar uma solução em partes menores, cada uma responsável por uma tarefa.



Pensamento computacional

Um dos pilares do pensamento computacional é a **decomposição**.

Decompor significa dividir um problema maior em problemas menores e mais administráveis.

Modularizar é aplicar decomposição ao programa.

Dividimos um problema maior em pequenas funções, cada uma responsável por uma tarefa específica.

Modularização

Modularizar significa organizar uma solução em partes menores, cada uma responsável por uma tarefa.

Programa grande

```c
#include <stdio.h>
int main() {
    // muitas linhas de código...
    return 0;
}
```

```c
cadastrar_aluno()
void cadastrar_aluno() {
    ...
}
```

```c
calcular_media()
float calcular_media() {
    ...
}
```

```c
verificar_frequencia()
int verificar_frequencia() {
    ...
}
```

```c
gerar_relatorio()
void gerar_relatorio() {
    ...
}
```

Benefícios

- Código mais organizado
- Maior reutilização
- Menos repetição
- Testes mais simples
- Manutenção facilitada
- Melhor compreensão da solução



Em vez de pensar em um programa gigante, pensamos em um conjunto de pequenas soluções que juntas resolver o problema computacional proposto.

Analogia da máquina de café



A pessoa não precisa conhecer todos os detalhes internos da máquina.

- quais dados fornecer;
- qual operação solicitar;
- qual resultado será produzido.

Máquina de café	Função
ingredientes/opções	parâmetros
processamento interno	corpo da função
bebida produzida	retorno
botão escolhido	chamada

Da máquina de café para uma função



Uma função encapsula uma tarefa e permite utilizá-la sem conhecer todos os detalhes de sua implementação.

Você já utiliza funções

Desde os primeiros programas utilizamos funções:

Saída na tela

```
printf("Olá!");
```

Entrada de dados

```
scanf("%d", &idade);
```

Abertura de arquivo

```
fopen("dados.txt", "r");
```

Fechamento de arquivo

```
fclose(arquivo);
```

E todo programa em C possui uma função especial:

```
int main(void)
{
    return 0;
}
```



`Então...` A novidade não está em `usar` funções!
Mas, aprender a `criar nossas próprias funções` e modularizar nossas implementações.

O que é uma função?

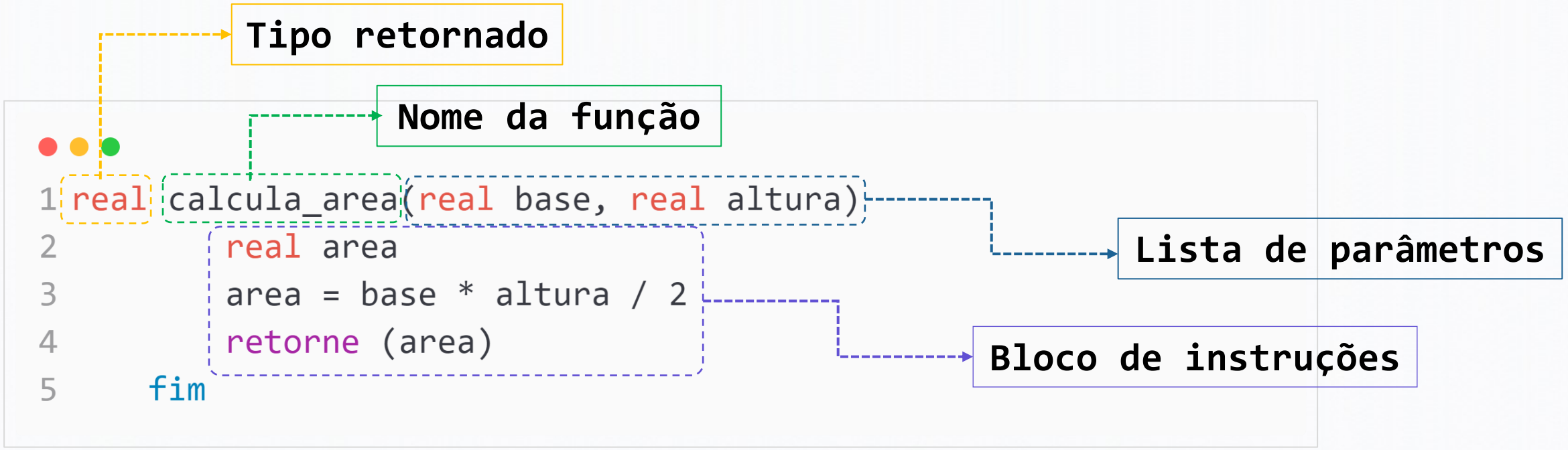
Uma **função** é um bloco de código identificado por um nome e responsável por realizar uma tarefa específica.

Uma função pode:

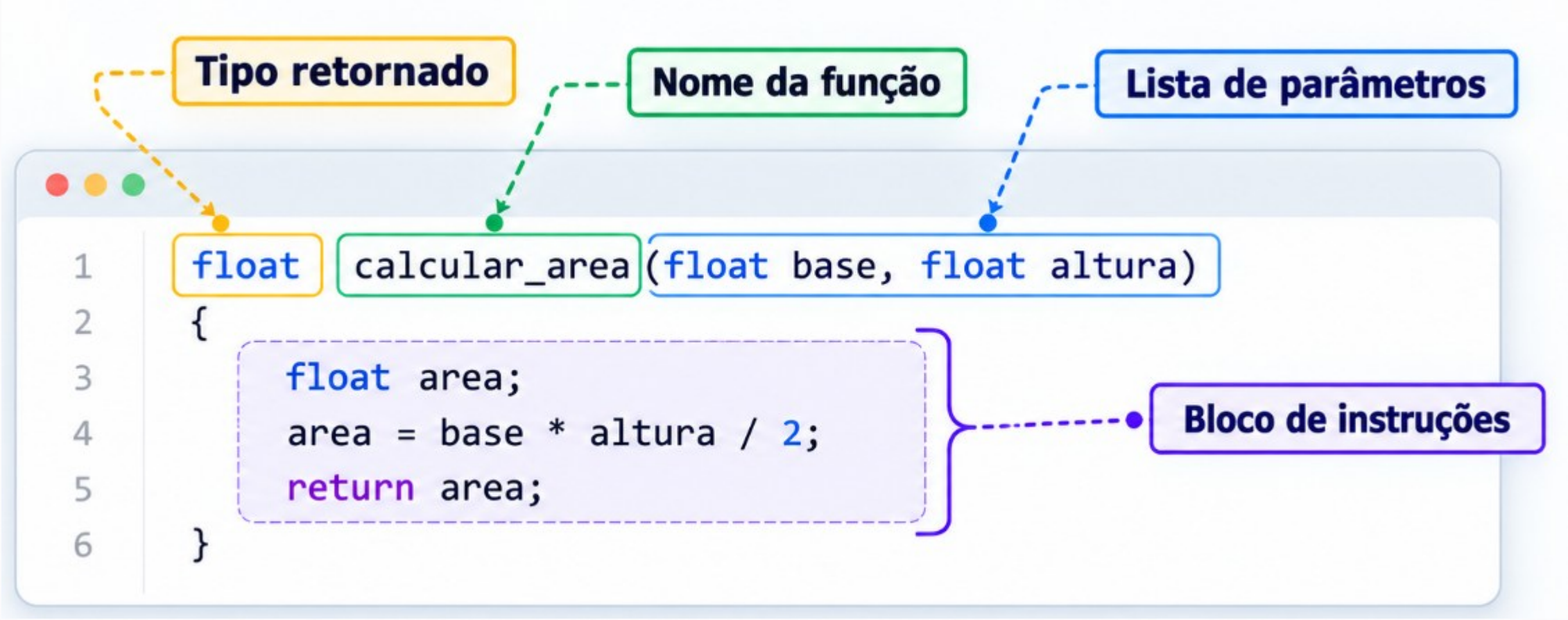
- Receber dados e parâmetros para realizar uma tarefa específica.
- Executar instruções para processar os dados recebidos
- Retornar um resultado (valor) após o processamento do dado recebido
- Ser reutilizada e chamada diversas vezes durante a execução do programa



Anatomia de uma função (algoritmo)



Anatomia de uma função em C



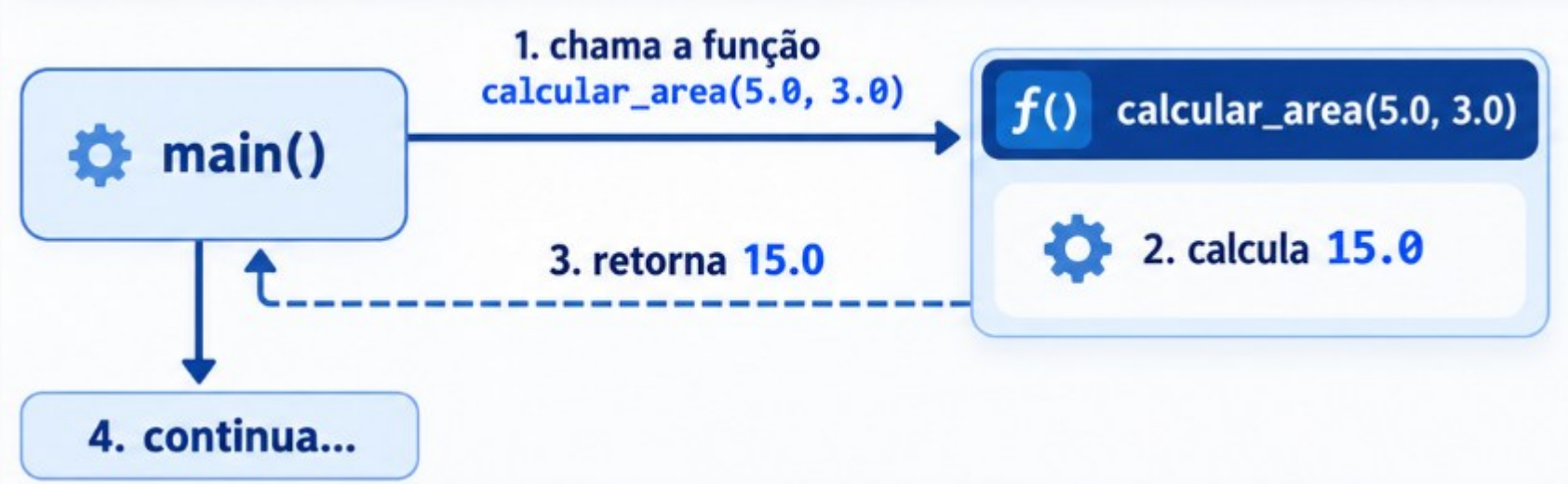
O bloco `{ ... }` contém as instruções executadas pela função.

Chamando uma função

Depois de criar uma função, precisamos **chamá-la.**

Code - exemplo.c

```c
area = calcular_area(5.0, 3.0);
```



Ao chamar uma função, o programa **desvia temporariamente o fluxo, executa a função** e depois **retorna** ao ponto da chamada.

Chamando uma função

Por que minha função não retorna nada???



Ah... não chamei a função 😬😬😂😂



Exercícios de aplicação



Uma transportadora calcula o valor do frete considerando o peso do pacote. Para determinado serviço, são cobrados R$ 4,50 por quilograma. Como esse cálculo será realizado muitas vezes pelo sistema, a regra deve ser implementada em uma função.

1- Desenvolva um programa que leia o peso do pacote e utilize a função:

```
float calcular_frete(float peso);
```

para calcular e retornar o valor do frete.

Caso de teste

```
Entrada:
Digite o peso (kg): 8.5

Saída:
Valor do frete: R$ 38.5
Deseja calcular outro frete (s/n)?
```

Funções sem retorno

Nem toda função precisa produzir um valor.

Uma função pode simplesmente **executar uma ação.**

Função sem retorno

```c
void exibir_linha(void)
{
    printf("--------------------------\n");
}
```

Quando não existe valor a retornar, utilizamos:

void

Chamando:

```c
exibir_linha();
```

Retome o exercícios 1 e crie uma função para exibir o relatório.

Argumentos × parâmetros

Na definição:

Code - exercicio1.c

```c
float calcular_frete(float peso);
```

peso é um **parâmetro.**

Na chamada:

Code - exercicio1.c

```c
frete = calcular_frete(peso);
```

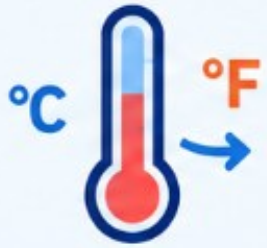
peso é um **argumento.**



Atividade rápida!!!!

Qual função você criaria para cada problema abaixo?

1 Converter Celsius para Fahrenheit



Entrada:
temperatura

Saída:
temperatura convertida

2 Mostrar um menu



Entrada:
nenhuma

Saída:
nenhuma

3 Determinar o maior entre dois valores



Entrada:
dois inteiros

Saída:
maior inteiro



Pense antes de escrever código:

- nome;
- parâmetros;
- tipo de retorno.

Parâmetros e ponteiros

Code - parametros_e_ponteiros.c

```c
#include <stdio.h>

void dobro(int numero){
    numero = numero*2;
}

int main()
{
    int numero;
    printf("Digite um numero inteiro: ");
    scanf("%d", &numero);

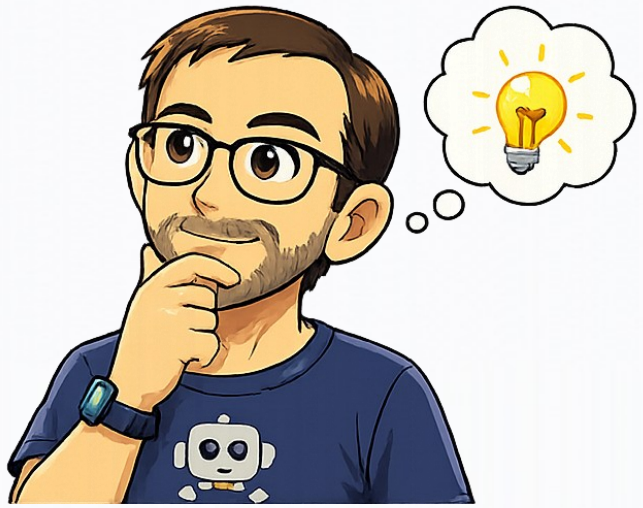
    dobro(numero);

    printf("O dobro do numero digitado eh: %d\n", numero);
    return 0;
}

```

Qual será a saída se o usuário digitar 5?

a) 5
b) 10
c) erro de compilação



O que aconteceu????

Quando fazemos a chamada **`dobro(numero)`**, a função **recebe uma cópia do valor.** A variável original **não é modificada**!!!

Programa principal (`main`)

Variável original:

```
numero = 5
```

A variável numero permanece com valor 5.

`dobro(numero)` passa uma cópia do valor 5

Função (`void dobro(int numero)`)

Variável local (parâmetro):

```
numero = 5
numero = numero *2
numero = 10
```

Apenas a variável local numero é modificada!

Consequências

- Em C, **os parâmetros recebem valores.**
- A função altera **somente a variável local.**
- Funções podem utilizar dados do programa principal **sem necessariamente modifica-los!**

Escopo de variável

O que é escopo de variável?

```c
void exemplo(void)
{
    int x = 10;
}

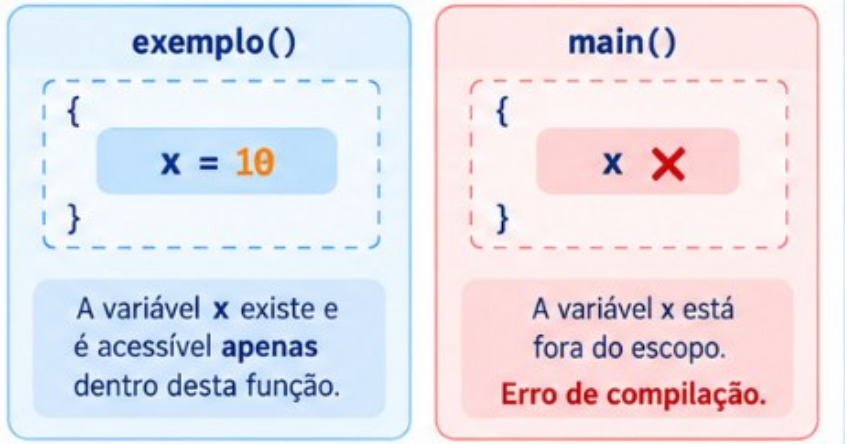
int main(void)
{
    printf("%d", x); // erro
}
```

A variável `x` existe apenas dentro da função `exemplo()`.

Variável local

Sua visibilidade está limitada ao bloco em que foi declarada.

Representação do escopo



Duas variáveis locais com o mesmo nome

```c
void funcao_a(void)
{
    int valor = 10;
}

void funcao_b(void)
{
    int valor = 20;
}
```

Essas duas variáveis: `valor` são **diferentes**.



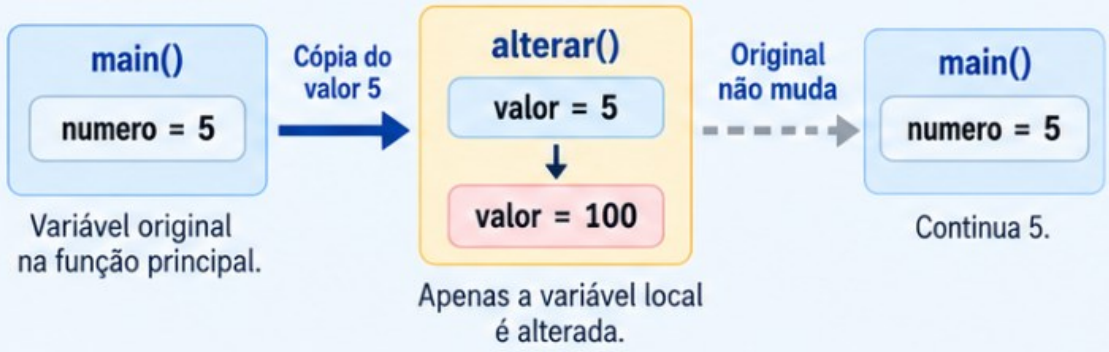
Variáveis locais existem apenas no bloco em que foram declaradas.

Nomes iguais em funções diferentes representam variáveis diferentes.

Parâmetro por valor x parâmetro por referência

Parâmetro por valor

- A função recebe uma **cópia do valor.**
- Alterações afetam **apenas a variável local.**
- A variável original **não é modificada.**




```
void alterar(int valor)
{
    valor = 100;
}
```

Por referência?

- Didaticamente, dizemos **passagem por referência.**
- Em C, o que é passado é o **endereço da variável.**
- Isso é feito com **ponteiros.**
- A função consegue **modificar a variável original.**




```
void alterar(int *valor)
{
    *valor = 100;
}
```

```
// Chamada da função
alterar(&numero);
```

Em C, **tudo é passado por valor.** Com **ponteiros e arrays**, o valor passado permite **acessar os dados originais.**

Exercícios de aplicação

Durante determinados processamentos, um programa precisa reorganizar valores. Uma operação fundamental consiste em trocar o conteúdo de duas variáveis.

2- Crie a função que receba os dois valores e troque seus conteúdos.

Caso de teste

Entrada:

```
Digite o valor de a: 10
Digite o valor de b: 20
```

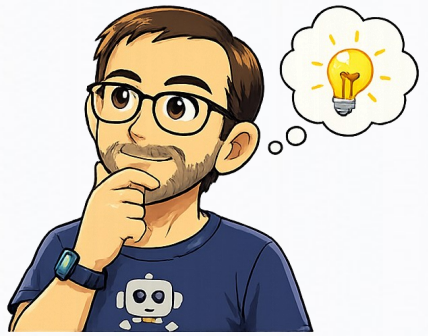
Saída:

```
Antes: a=10 b=20
Depois: a= 20 b=10
```

Neste caso precisamos utilizar ponteiros para alterar o valor da variável

Code - parametros_e_ponteiros.c

```c
void trocar(int *a, int *b){
    int auxiliar;
    auxiliar = *a;
    *a = *b;
    *b = auxiliar;
}
```



Em C, **todos os argumentos são passados por valor.**

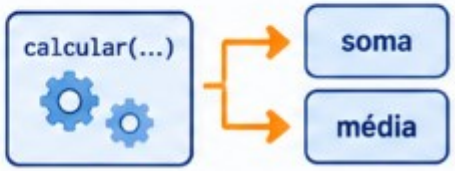
Por que usar ponteiros em parâmetros?

Eles permitem que uma função:

1. altere variáveis do chamador;



2. produza mais de um **resultado**;

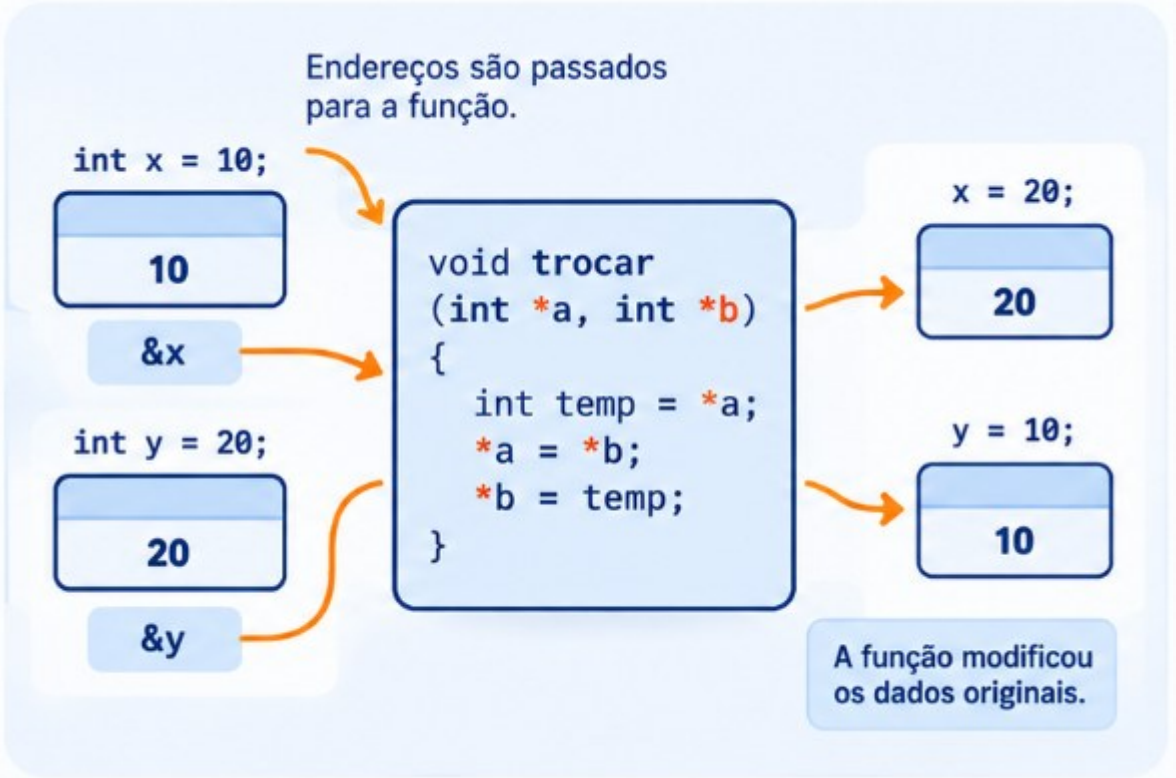


3. manipule estruturas maiores sem criar cópias desnecessárias;



4. trabalhe com vetores, **strings** e **estruturas de dados**.





Esse mecanismo é recorrente em C.

Uma situação diferente: arrays

1 Exemplo do código

função alterar

```
void alterar(int vetor[])
{
    vetor[0] = 100;
}
```

código principal

```
int numeros[3] = {10, 20, 30};

alterar(numeros);

printf("%d", numeros[0]);
```

✓ **Resultado: 100**

? **Por que o vetor original mudou se não utilizamos &numeros?**

2 Arrays em parâmetros de função

Quando um array é utilizado como parâmetro de função, ele é tratado **essencialmente como um ponteiro para seus elementos.**

`void alterar(int vetor[])` ⟷ `void alterar(int *vetor)`

Representam a mesma ideia nesse contexto.

`vetor[]`

↓

endereço do primeiro elemento

3 O que realmente é enviado?

Vetor original: numeros

[0]	[1]	[2]
10	20	30

endereço do primeiro elemento

Função alterar

```
void alterar(int vetor[])
{
    vetor[0] = 100;
}
```

Vetor atualizado: numeros

[0]	[1]	[2]
100	20	30

✓ Ao chamar a função, o programa envia acesso aos elementos do vetor. Por isso, a função **modifica o vetor original.**

Em parâmetros de função, os `arrays` permite acesso direto aos elementos originais (**passagem por referência**).

Protótipo, chamada e implementação

O compilador precisa conhecer uma função antes que ela seja utilizada.

O protótipo funciona como um aviso antecipado ao compilador

```
float calcular_area(float largura, float altura);
```

É como dizer: existe uma função chamada **calcular_area**, ela recebe dois float e retorna um float.
A implementação aparecerá depois.

Por que utilizar o protótipo?

- O compilador precisa conhecer a função antes que ela seja utilizada.
- O protótipo informa:
  - nome
  - quantidade de parâmetros
  - tipos dos parâmetros
  - tipo retornado
- O protótipo não executa a função. Ele apenas informa ao compilador que ela existe.

exemplo.c

```c
// Protótipo
float calcular_area(float largura, float altura);

int main(void)
{
    float resultado;

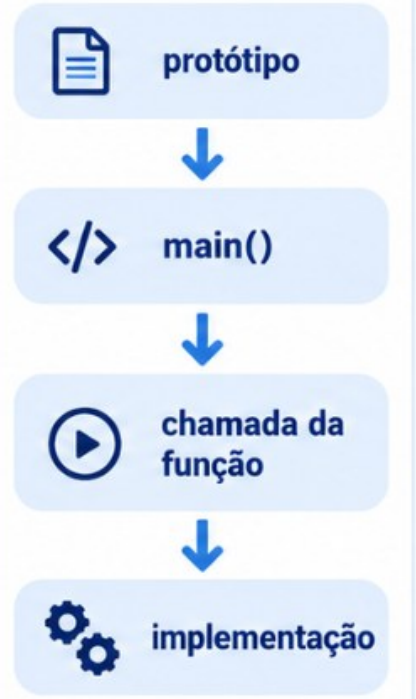
    resultado = calcular_area(5.0, 3.0);

    printf("Área = %.2f\n", resultado);

    return 0;
}

// Implementação
float calcular_area(float largura, float altura)
{
    return largura * altura;
}
```

Fluxo



Em C, não é obrigatório escrever um protótipo separado para toda função. **Porém, em C moderno, não é uma boa prática chamar a função antes dela ter sido declarada.**

Vamos praticar





Exercícios de aplicação



Uma escola precisa de um sistema que auxilie os professores a calcular a média de notas dos alunos e determinar automaticamente se cada aluno está aprovado ou reprovado. Para ser aprovado, o aluno deve ter uma média igual ou superior a 7.

3- Desenvolva um programa em C que solicite duas notas de um aluno, calcule a média e informe o resultado, utilizando funções. Permita que o programa seja executado enquanto o usuário possuir notas a serem digitada!

Caso de teste

```
Entrada:
Digite a primeira nota: 7
Digite a segunda nota: 3

Saída:
Média do aluno: 5.0
Situação: Reprovado
Deseja digitar notas de outro aluno (s/n)?
```

Exercícios de aplicação



Uma empresa de climatização precisa calcular frequentemente a área de ambientes retangulares para estimar a quantidade de piso necessária. Como esse cálculo será reutilizado diversas vezes, decidiu-se implementá-lo em uma função.

4- Desenvolva um programa em C que:

- leia a largura (em metros);
- leia o comprimento (em metros);
- utilize uma função `calcularArea()` para calcular a área;
- exiba o resultado;
- permita calcular quantas áreas o usuário precisar.

Caso de teste

```
Entrada:
Digite a largura (m): 4.5
Digite o comprimento (m): 8

Saída:
Área: 36.0 m²
Deseja calcular outra área (s/n)?
```

Um novo problema!!!

Já sabemos como armazenar vários valores do mesmo tipo em um array:

```
float notas[10];
```

1 Arrays são estruturas homogêneas

```
float notas[5];
```

float	float	float	float	float

Todos os elementos possuem o **mesmo tipo.**

2 Mas como representar um aluno?

Aluno

nome	→	texto
rgm	→	inteiro
nota	→	real
frequência	→	real

Os dados possuem **tipos diferentes,** mas pertencem à mesma entidade.

? E se precisarmos reunir... `int` `char[]` `float` na mesma estrutura?

O que é uma struct?

Uma **struct** permite **agrupar dados de diferentes tipos em uma única estrutura.**

Declaração da struct

```
struct aluno
{
    char nome[50];
    int rgm;
    float media;
};
```

Agora podemos declarar:

```
struct aluno aluno1;
```

Representando uma entidade

aluno1		
nome	→	"Ana"
rgm	→	12345
media	→	8.5

Uma struct modela um **registro** composto por vários atributos relacionados.

Utilizamos o operador ponto:

```
aluno1.rgm = 12345;
aluno1.media = 8.5;
```

Exibição:

```
printf("%d\n", aluno1.rgm);
printf("%.2f\n", aluno1.media);
```

Sintaxe:

`variavel.campo`

typedef para simplificar

Podemos escrever um typedef para uma struct para simplificar suas declarações.

```
typedef struct
{
    char  nome[50];
    int   rgm;
    float media;
} Aluno;
```

typedef cria um apelido para o tipo, deixando a declaração mais simples e legível.

Agora podemos declarar:

```
Aluno aluno1;
```

Mais simples e legível!

em vez de:

```
struct aluno aluno1;
```

Mais verboso

Funções também podem receber *structs*

1 Passando uma struct por valor

```c
void exibir_aluno(Aluno aluno)
{
    printf("Nome: %s\n", aluno.nome);
    printf("RGM: %d\n", aluno.rgm);
    printf("Média: %.2f\n", aluno.media);
}
exibir_aluno(aluno1);
```



- A função recebe uma **cópia da estrutura.**
- Para pequenas estruturas isso pode ser aceitável.
- Para estruturas maiores, copiar todos os dados pode ser desnecessário.

2 Passando o endereço com ponteiros

```c
void atualizar_media(Aluno *aluno)
{
    aluno->media = 8.5;
}

atualizar_media(&aluno1);
```



- Podemos enviar seu **endereço** utilizando **ponteiros**.
- Nesse caso, a função acessa a **estrutura original**.

Por valor: recebe uma cópia.

Com ponteiro: acessa a estrutura original.

Esse padrão será muito útil em C quando trabalharmos com estruturas maiores.

Vamos praticar





Exercícios de aplicação

Um sistema acadêmico precisa armazenar o RGM e duas notas de um estudante e calcular sua média.

5- Crie uma estrutura **`Aluno`** contendo:

```
rgm
nota1
nota2
media
```

Implemente:

```
void calcular_media(Aluno *aluno);
```

A função deverá atualizar o campo **`media`**.

Caso de teste

```
Entrada:
RGM: 123456
Nota 1: 7.0
Nota 2: 9.0

Saída:
RGM: 123456
Média: 8.00
```

Exercícios de aplicação

Um sistema acadêmico precisa armazenar o RGM e duas notas de um estudante e calcular sua média.

6- O programa deverá armazenar os dados de 3 alunos em um vetor de estruturas. Para cada aluno:

- leia o RGM;
- leia a primeira nota;
- leia a segunda nota;
- calcule a média utilizando a função:

```
void calcular_media(Aluno *aluno);
```

utura correspondente ao aluno e atualizar o campo `media`. Depois de cadastrar todos os alunos, o programa deverá solicitar um **RGM para consulta** exibir (caso encontrado) o RGM e a média. Caso contrário exibir uma mensagem de erro.

Desafio: Sistema acadêmico com menu



O sistema acadêmico precisa permitir que o usuário cadastre alunos e faça consultas sem precisar encerrar e executar novamente o programa. Para isso, será utilizado um menu.

7- Crie um programa que apresente continuamente o seguinte menu:

```
===== SISTEMA ACADÊMICO =====

1 - Cadastrar aluno
2 - Consultar aluno por RGM
3 - Listar alunos
4 - Sair

Escolha uma opção:
```

O programa deverá armazenar até **10 alunos** e utilizar funções para cadastro, consulta, listagem de alunos e cálculo da média.

Desafio: Sistema acadêmico com menu



O sistema acadêmico precisa permitir que o usuário cadastre alunos e faça consultas sem precisar encerrar e executar novamente o programa. Para isso, será utilizado um menu.

Para aumentar ainda mais a modularização, vamos criar as seguintes funções:

```c
void calcular_media(Aluno *aluno);

void cadastrar_aluno(Aluno alunos[], int *quantidade);

void consultar_aluno(Aluno alunos[], int quantidade);

void listar_alunos(Aluno alunos[], int quantidade);
```

Observe que em parâmetros de função, **arrays já são tratados essencialmente como ponteiros para seus elementos**. Poderíamos utilizar também `(Aluno *alunos, int *quantidade)`

Alguma dúvida????



Universidade Cruzeiro do Sul

CRÉDITOS

Material elaborado por:
Prof. Marco Antonio Sanches

Última atualização: 2026/2

CONTATOS


marco.sanches@cruzeirodosul.edu.br


marco-antonio-sanches-anastácio-601b9123